



Attachment input

Turn file state into clear attachment controls.

Problem and boundary

Public contract

Working integration

@epam/ai-dial-attachment-input



When this library is useful

Turn file state into clear attachment controls.

01

Compose with files

Show files before
sending a message.

02

Recover from failure

Expose remove and
retry actions per
attachment.

03

Reuse sent-file UI

Render image groups
and file rows in
transcripts.



Follow the ownership boundary

Inputs and effects stay explicit.



Library: Cards, trays, file-drop affordances and clipboard handling.



The public contract to start with

AttachmentTray · selected surface; full API index is in the bundle.

Input or operation	Contract
attachments	DisplayAttachment[]
onRemove (optional)	((id: string) => void)
onRetry (optional)	((id: string) => void)
labels (optional)	AttachmentTrayLabels



A minimal workspace integration

TypeScript / React example · imports and host setup abridged.

```
function AttachmentDemo({ initial }: { initial: DisplayAttachment[] }) {  
  const [attachments, setAttachments] = useState(initial);  
  return <AttachmentTray attachments={attachments}  
    onRemove={id => setAttachments(items => items.filter(x => x.id !== id))}  
    labels={{ ariaLabel: 'Attached demo files' }} />;  
}
```

Full example: [examples/attachment-input.tsx](#) · uses [@epam/ai-dial-attachment-input](#)



Wire a realistic host workflow

Keep attachment input focused on its public responsibility.

01

Accept a file

The host starts an upload and creates display data.

02

Reflect progress

Pass current attachments and their upload state into the tray.

03

Retry or remove

Use the attachment id from the callback to update host state.



Available customization

01

Tray and group

Use AttachmentTray while composing; AttachmentGroup for sent files.

02

Lazy previews

Image loading has an observable loading and error lifecycle.

03

Accessible actions

Supply localized open, retry, remove and upload labels.



Find the integration and its checks

Existing tests are evidence of intended behavior; suite results are not claimed.

Evidence	Where to look
Real consumer	conversation-messages/src/components/MessageBubble/ UserMessageBubble.tsx
Test evidence	handles null clipboardData without throwing
Test evidence	starts in loading state when enabled with a source



Constraints that affect integration

01

No uploader in a card

Showing progress does not perform the network upload.

02

Watch callback shapes

Tray actions use an id; message bubble callbacks can use an object.

03

Keep state ownership clear

The library has local UI state; the host owns durable file state.